



FACULTAD DE
INTELIGENCIA ARTIFICIAL
E INGENIERIAS

INFORME FINAL

PROYECTO K-QGMIP

Extensión de QNodes y GeoMIP a k-particiones

Integrantes:

Mauricio Gonzales Carmona
Manuela Vallejo Toro

Universidad de Caldas

Docente:

Luz Enith Guerrero Mendieta

Manizales, junio de 2026

Índice

1. Objetivo	3
2. Introducción	3
3. Marco teórico	3
3.1. Matriz de probabilidad de transición (TPM)	3
3.2. Tensores, N-Cubos y producto tensorial	3
3.3. Presente, futuro y subsistema	4
3.4. Pérdida de información: EMD efecto	4
3.5. Independencia entre valores de k	4
4. Punto de partida: QNodes y GeoMIP base	5
4.1. QNodes (KQNodes) — $k = 2$	5
4.2. GeometricSIA (KGeoMIP) — $k = 2$	5
5. Modificaciones e implementación del proyecto K-QGMIP	5
5.1. Componente principal: KPartitionSIA	5
5.2. Archivos nuevos	6
5.3. Archivos modificados	6
5.4. Bugs corregidos	6
6. Heurísticas y algoritmos de aproximación	7
6.1. Heurísticas implementadas en KPartitionSIA	7
6.2. Selección de estrategia según tamaño de red	7
7. Monte Carlo y MCTS	7
7.1. Motivación	7
7.2. Estimador MC-EMD	7
7.3. MCTS	8
8. Procedimiento experimental	8
8.1. Dataset y casos	8
8.2. Pipelines ejecutados	8
8.3. Entorno	9
9. Resultados y análisis	9
9.1. Cobertura consolidada (junio 2026)	9
9.2. Hallazgos cuantitativos	9
9.3. Gráficas del proyecto	10
10. Pseudocódigo	12
10.1. API principal: <code>find_k_partition</code>	12
10.2. Evaluación de partición	12
10.3. MC-EMD	12
10.4. MCTS	13
10.5. QNodes (heurística Q, $k=2$)	13
11. Limitaciones	13

12.Conclusiones	14
Referencias del repositorio	14

1. Objetivo

Extender los algoritmos **QNodes** (KQNodes) y **GeoMIP** (KGeoMIP), originalmente diseñados para biparticiones ($k = 2$), hacia **k-particiones** con $k \in \{2, 3, 4, 5\}$ sobre las redes de prueba **DatosPruebas2026**. El objetivo es minimizar la pérdida de información causal ϕ al dividir el mecanismo en k grupos independientes y recombinar sus dinámicas mediante producto tensorial, midiendo la distancia con **EMD** (Earth Mover's Distance). Se implementan heurísticas exactas y aproximadas, se documenta la frontera computacional en redes grandes ($n \geq 20$) y se contrastan resultados exactos frente a métodos Monte Carlo en el benchmark rápido.

2. Introducción

En el marco de la *Integrated Information Theory* (IIT), un sistema dinámico de variables binarias se modela mediante una **matriz de probabilidad de transición** (TPM) que codifica la evolución temporal. Al analizar un **subsistema** —definido por un purview (alcance, futuro) y un mecanismo (presente)— surge la pregunta central del proyecto: si se parte el subsistema en k componentes independientes y se reconstruye la dinámica por producto tensorial, ¿cuánta información causal se pierde?

Esa pérdida se denota ϕ y se calcula con EMD usando la distancia de **Hamming** como métrica de transporte entre configuraciones del hipercubo. Un valor $\phi = 0$ indica que la partición preserva por completo las dependencias relevantes; $\phi > 0$ cuantifica su destrucción.

El repositorio base del curso ya contenía implementaciones maduras de bipartición: **QNodes** (heurística Q submodular) y **GeometricSIA** (programación dinámica geométrica, MIP exacto en $k = 2$). Nuestro aporte consiste en la clase **KPartitionSIA**, los pipelines de benchmark (exacto, rápido y QNodes dedicado), la estimación **MC-EMD** y la búsqueda **MCTS** para redes donde el costo $O(2^n)$ vuelve inviable el método exacto.

3. Marco teórico

3.1. Matriz de probabilidad de transición (TPM)

Para un sistema de n nodos binarios, la TPM tiene 2^n filas (estados globales en $t+1$) y n columnas. Cada entrada $TPM[i, j]$ representa la probabilidad condicional de que el nodo j adopte un estado dado el contexto del sistema. En los archivos del proyecto (**N{n}A.csv**) las filas corresponden a todos los estados posibles y el separador es `:`.

Para $n = 25$ se tienen $2^{25} \approx 33,5 \times 10^6$ filas; en memoria, con precisión `float32`, la matriz ocupa $\approx 3,2$ GB.

3.2. Tensores, N-Cubos y producto tensorial

Cada nodo del subsistema posee un **N-Cubo**: un arreglo de forma $(2, 2, \dots, 2)$ cuyas entradas almacenan probabilidades conjuntas sobre configuraciones ON/OFF. Al **marginalizar** dimensiones del mecanismo se obtienen distribuciones sobre el purview (futuro).

Si las partes de una k -partición son condicionalmente independientes, la distribución conjunta reconstruida es el **producto de Kronecker** de las marginales unidimensionales:

$$p_{\text{joint}} = p_1 \otimes p_2 \otimes \cdots \otimes p_n, \quad p_i = [p_{\text{OFF}}, 1 - p_{\text{OFF}}]. \quad (1)$$

En código, esto se implementa con `np.kron` en la función `_reconstruir_tensor` de `kpartition.py`.

3.3. Presente, futuro y subsistema

Cada caso de prueba especifica:

- **Condición:** nodos fijados en background.
- **Alcance (purview):** nodos activos en el futuro ($t+1$).
- **Mecanismo:** nodos activos en el presente (t).

El pipeline `sia_preparar_subsistema()` condiciona la TPM, extrae el subsistema y calcula las marginales de referencia `sia_dists_marginales`. Este paso se ejecuta **una vez por caso** y se reutiliza entre estrategias mediante caché (`_SUBSYSTEM_CACHE`).

El tamaño efectivo del problema lo determina $|\text{alcance} \cap \text{mecanismo}|$ y el número de nodos del subsistema resultante, **no** el n nominal del sistema completo. Por ello muchos casos de $N=25$ operan sobre subsistemas de $n \approx 11-15$ y terminan en segundos, mientras que los pocos con subsistema casi completo dominan el tiempo total.

3.4. Pérdida de información: EMD efecto

La métrica principal del benchmark es la **EMD efecto** sobre marginales unidimensionales:

$$\phi_{\text{efecto}} = \sum_i |u_i - v_i|, \quad (2)$$

donde u_i y v_i son las probabilidades de que el nodo i esté en OFF bajo la distribución particionada y la de referencia, respectivamente. Para evaluar una k -partición, `_evaluar_particion` marginaliza cada parte, compara con la referencia y suma las contribuciones.

En el algoritmo geométrico, la tabla de costos de transición usa el factor de decrecimiento

$$\gamma = 2^{-\text{Hamming}(s_i, s_j)}, \quad (3)$$

que pondera la similitud entre estados del mecanismo según su distancia de Hamming.

3.5. Independencia entre valores de k

Cada $k \in \{3, 4, 5\}$ se resuelve **desde cero**: Greedy o Kernighan–Lin generan una asignación inicial y refinan con búsqueda local, sin tomar la partición óptima de $k-1$. Por tanto:

- No hay monotonía garantizada: puede ocurrir $\phi(k=4) > \phi(k=3)$ en un mismo caso.
- Los tiempos de $k=3, 4, 5$ son del mismo orden en un subsistema dado, pues cada búsqueda repite trabajo similar.

4. Punto de partida: QNodes y GeoMIP base

4.1. QNodes (KQNodes) — $k = 2$

Archivo base: `controllers/strategies/q_nodes.py`.

Implementa la heurística **Q submodular** del repositorio QNodes del curso:

1. Construye vértices (tiempo, índice) para presente y futuro.
2. Crece incrementalmente el conjunto ω eligiendo el δ que minimiza la ganancia submodular basada en EMD.
3. Memoiza evaluaciones en `memoria_omega` y `memoria_particiones`.

Complejidad aproximada $O(N^3 \cdot 2^n)$ con N nodos del subsistema. Es referencia combinatoria del enunciado; en casos ligeros coincide con GeoMIP.

4.2. GeometricSIA (KGeoMIP) — $k = 2$

Archivo base: `controllers/strategies/geometric.py`.

1. Ejecuta `find_mip()`: BFS que llena `tabla_transiciones` con costos entre estados del mecanismo.
2. Genera candidatos de bipartición con `identificar_particiones_optimas()`.
3. Evalúa cada candidato con `bipartir + emd_efecto`.
4. Retorna la partición de **mínima pérdida** (óptimo global en $k=2$ para el modelo).

El cuello de botella es `find_mip`, que escala como $O(n \cdot 2^n)$ en tiempo y memoria.

5. Modificaciones e implementación del proyecto K-QGMIP

5.1. Componente principal: KPartitionSIA

Archivo nuevo: `kpartition.py`. Expone la API del enunciado:

`find_k_partition(condicion, alcance, mecanismo, tpm) → Solution`.

Para $k \geq 3$ no existe MIP exacto intratable; se implementan heurísticas que compiten por menor ϕ .

5.2. Archivos nuevos

Cuadro 1: Principales archivos añadidos al repositorio.

Archivo	Función
kpartition.py	k-particiones, Greedy, KL, MCTS, MC-EMD
benchmark.py	Benchmark exacto DatosPruebas2026
benchmark_rapido.py	MCTS + MC-EMD
benchmark_aprox.py	KL + MC-EMD aproximado
run_qnodes_k2.py	QNodes $k=2$ con checkpoints
gen_comparativa.py	Tablas exacto vs. rápido
tpm_io.py	Carga streaming float32 ($n \geq 21$)
tests/ (48 tests)	Coherencia, EMD, KL, memo QNodes

5.3. Archivos modificados

- sia.py: caché de subsistemas `_SUBSYSTEM_CACHE`.
- geometric.py: caché `find_mip`, modo silencioso.
- solution.py: campos `n_nodos`, `k`.
- q_nodes.py: memoización `memoria_omega` (bitácora 37).

5.4. Bugs corregidos

Cuadro 2: Correcciones relevantes durante el desarrollo.

Bug	Efecto	Solución
<code>_refinar_kl</code> duplicaba partes	Más grupos que k en Excel	Corregir índices al mover nodos
MCTS omitía partes vacías	k incorrecto en salida	Incluir partes vacías
<code>merge_with_previous</code> con <code>.update()</code>	Filas perdidas QNodes $n=22$	<code>concat</code> + <code>drop_duplicates</code>

Se eliminó el módulo duplicado **Method3**; los resultados quedaron centralizados en `GeoMIP/data/results/`.

6. Heurísticas y algoritmos de aproximación

6.1. Heurísticas implementadas en KPartitionSIA

Cuadro 3: Estrategias de k-partición implementadas.

Heurística	k	Descripción
GeometricSIA	2	MIP exacto (delegación desde KPartitionSIA)
QNodes	2	Heurística Q submodular
Greedy	3–5	Extiende bipartición MIP dividiendo nodos en nuevas partes
Kernighan–Lin (KL)	3–5	Búsqueda local: mover nodos entre partes
Clustering Ward	3–5	Sobre matriz de costos aumentada
Spectral + EMD	3–5	Afinidad $C[i] \cdot C[j]$ / normalización
MCTS + MC-EMD	2–5	Árbol UCB; benchmark rápido
KL + MC-EMD	2–5	KL con evaluación Monte Carlo; $n=25$ rápido

6.2. Selección de estrategia según tamaño de red

Cuadro 4: Modo de ejecución según n y pipeline.

n	Pipeline	k	Método principal
10, 15, 20	Exacto	2	Geo + QNodes
10, 15, 20, 22	Exacto	3–5	Greedy + KL + <code>find_mip</code>
22, 25	Rápido	2–5	MCTS + MC-EMD ($n \leq 22$)
25	Rápido	2–5	KL + MC-EMD
22, 25	QNodes dedicado	2	Heurística Q (parcial)

7. Monte Carlo y MCTS

7.1. Motivación

Marginalizar exactamente sobre el mecanismo cuesta $O(2^{n_{\text{mec}}})$ accesos al N-Cubo. En subsistemas con $n_{\text{mec}} \geq 17$ cada evaluación de Greedy/KL supera los timeouts del benchmark. Por ello se añadieron dos capas (bitácora 23):

1. **MC-EMD:** estimación de ϕ muestreando S estados aleatorios del mecanismo.
2. **MCTS:** búsqueda en árbol sobre asignaciones k -partición con exploración UCB1.

7.2. Estimador MC-EMD

En lugar de promediar $2^{n_{\text{mec}}}$ estados:

1. Se muestrean S configuraciones aleatorias (típicamente $S = 2500\text{--}3000$).
2. Se hace *lookup* vectorizado en `NCube.data`.
3. Se promedian las contribuciones $\rightarrow$ marginal estimada por parte.
4. Se calcula ϕ con `emd.efecto`.

El error estadístico es del orden $O(1/\sqrt{S})$ (estimador insesgado). El *speedup* frente al exacto es aproximadamente $2^{n_{\text{mec}}}/S$. Para $n_{\text{mec}} = 20$ y $S = 3000$ se obtiene un factor $\sim 350\times$.

7.3. MCTS

- **Nodo del árbol:** asignación de etiquetas $1..k$ a nodos del subsistema.
- **Acción:** mover un nodo de una parte a otra.
- **Recompensa:** $-\phi$ (MCTS maximiza, equivalente a minimizar pérdida).
- **Exploración:** UCB1 con $c \approx \sqrt{2}$:

$$\text{UCB}(i) = Q_i + c\sqrt{\frac{\ln N_{\text{padre}}}{N_i}}. \quad (4)$$

En el benchmark rápido, los rollouts evalúan ϕ con MC-EMD. Para $n = 25$, MCTS resultó demasiado lento en $k=2$; se substituyó por **KL + MC-EMD** (bitácora 35), completando 50/50 casos.

8. Procedimiento experimental

8.1. Dataset y casos

Se utilizó el dataset oficial **DatosPruebas2026**: 50 casos por red ($n \in \{10, 15, 20, 22, 25\}$), cada uno con par (Purview, Mecanismo) codificado en letras y convertido a cadenas binarias con `letters_to_binary`.

8.2. Pipelines ejecutados

1. **Benchmark exacto** (`benchmark.py`): Geo $k=2$, Greedy y KL para $k=3, 4, 5$; QNodes $k=2$ solo en $n \leq 15$ integrado al flujo principal.
2. **QNodes dedicado** (`run_qnodes_k2.py`): $n = 22, 25$ con timeouts de hasta 24 h por caso y checkpoints cada 5 casos.
3. **Benchmark rápido** (`benchmark_rapido.py`): MCTS + MC-EMD ($n \leq 22$), KL + MC-EMD ($n = 25$), timeout 300 s por estrategia.
4. **Comparativa** (`gen_comparativa.py`): por cada red n , genera `n{n}_comparativa.xlsx` (formato ancho) y actualiza `comparativa_long.csv`.
5. **Gráficas** (`generar_graficas_proyecto.py`): una figura por matriz en `outputs/plots/proyecto`, (5 archivos para $n = 10, 15, 20, 22, 25$).

8.3. Entorno

- Python 3.12, gestor `uv`, entorno `~/venvs/kqgmip`.
- Corridas largas en WSL (Ubuntu); TPM $N=25$ cargada en streaming `float32`.
- 48 tests con `pytest` para validar coherencia $k=2$, EMD, KL y memoización QNodes.

9. Resultados y análisis

9.1. Cobertura consolidada (junio 2026)

Cuadro 5: Estado de los benchmarks por tamaño de red.

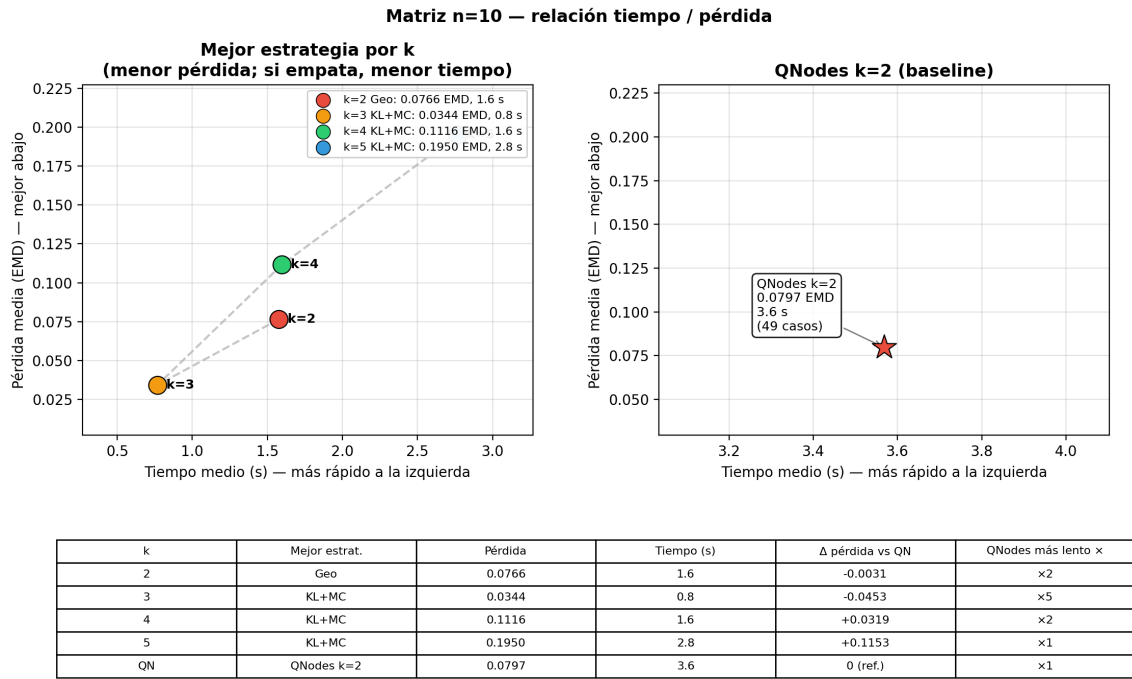
Modo	n	Casos	Notas
Exacto Geo $k=2$	10, 15, 20, 22	50/50	Referencia mayo–junio 2026
Exacto QNodes $k=2$	10, 15, 20	50/50	Coincide con Geo en casos ejecutados
Exacto QNodes $k=2$	22	42/50	8 casos pesados omitidos
Exacto QNodes $k=2$	25	33/50	17 casos pesados omitidos
Exacto Greedy/KL $k=3,5$	10, 15, 20, 22	50/50	Heurístico; no óptimo global
Rápido MCTS/KL-MC	10–25	50/50	Aproximado; post-fix junio 2026

Los casos omitidos en QNodes ($n=22$: #16–18, #22–25, #50; $n=25$: además #1–4, #8–11, #15) corresponden a subsistemas casi completos donde el algoritmo Q requiere **horas por caso**, no a fallos de memoria en la carga de la TPM.

9.2. Hallazgos cuantitativos

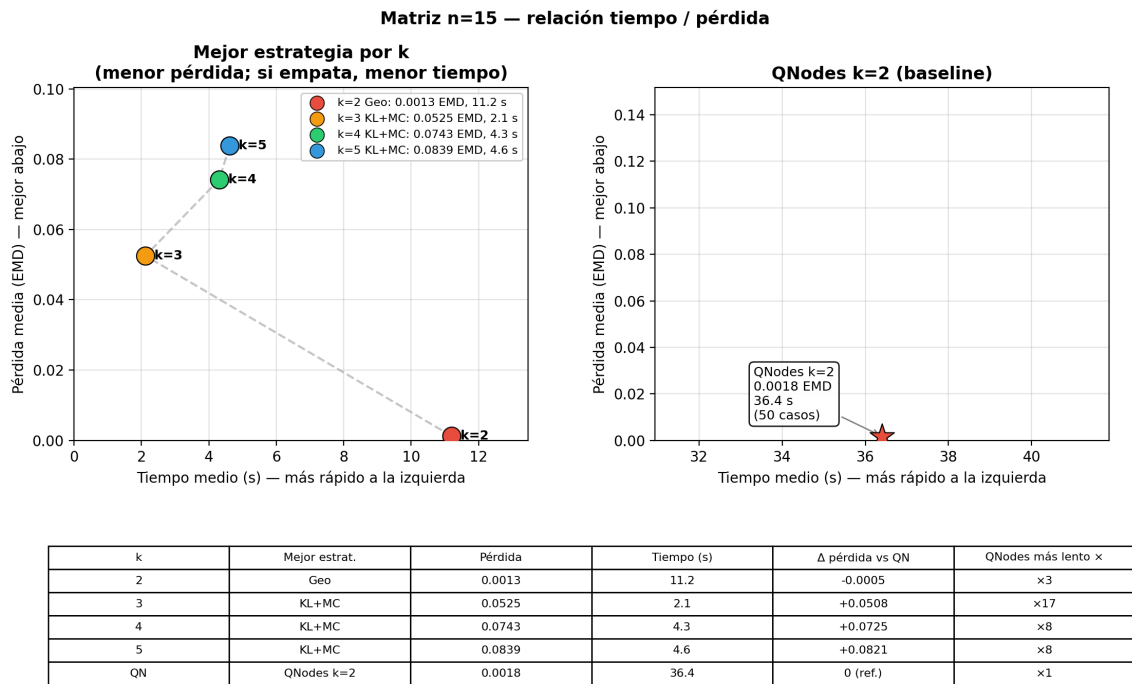
1. En $n = 20$, Geo y QNodes exactos alcanzan $\phi = 0$ en la mayoría de casos ligeros.
2. Geo $k=2$ en $n = 20$ puede tardar ~ 14 min por caso pesado; Greedy/KL en $k \geq 3$ terminan en segundos para el mismo subsistema.
3. El benchmark rápido en $n = 25$ reporta ϕ mayor que el exacto (cuando existe), pero en tiempo viable (~ 10 s–6 min por caso según tamaño del subsistema).
4. En los 42 casos QNodes $n=22$ ejecutados, la partición y ϕ coinciden con Geo $k=2$ (validación bitácara 40).

9.3. Gráficas del proyecto



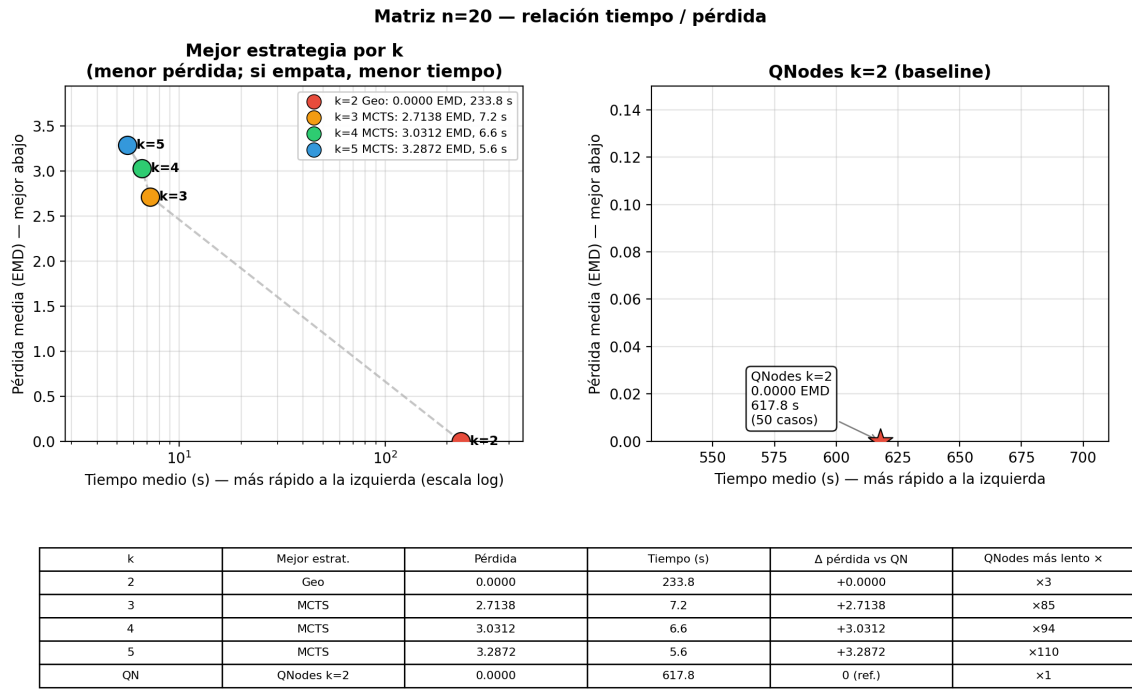
Fuente: n10_comparativa.xlsx · Paneles con ejes independientes (izq. heurísticas k=2...5, der. QNodes k=2)

Figura 1: Matriz $n=10$: mejor heurística por k (izq.) y QNodes $k=2$ (der.).



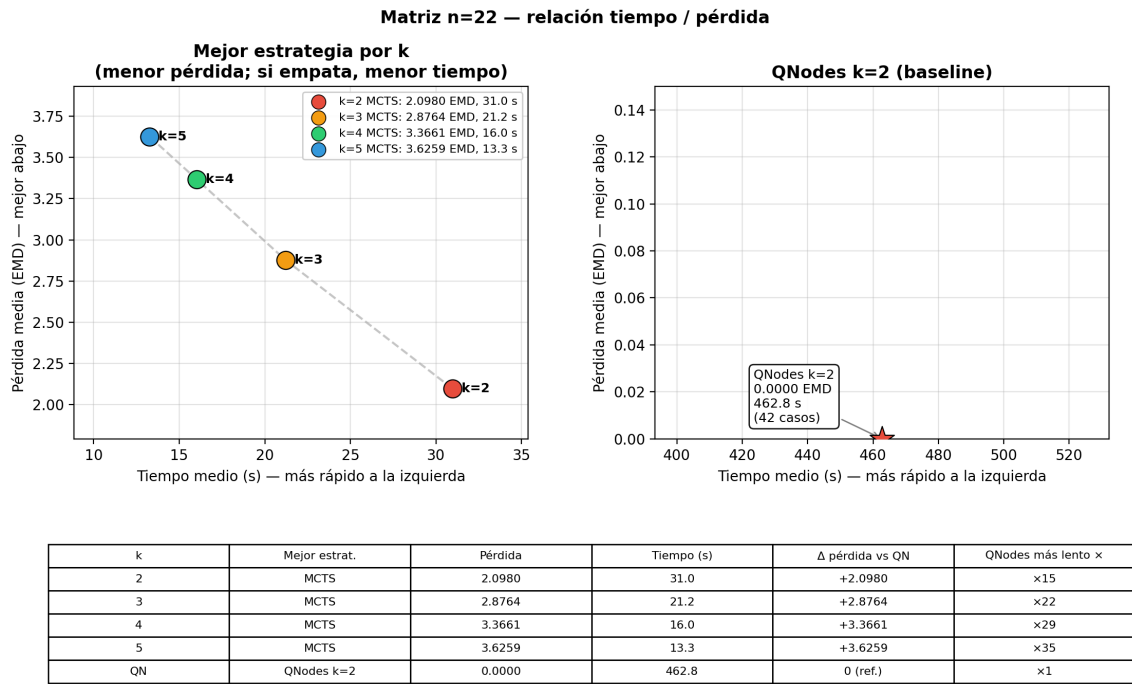
Fuente: n15_comparativa.xlsx · Paneles con ejes independientes (izq. heurísticas k=2...5, der. QNodes k=2)

Figura 2: Matriz $n=15$: mejor heurística por k (izq.) y QNodes $k=2$ (der.).



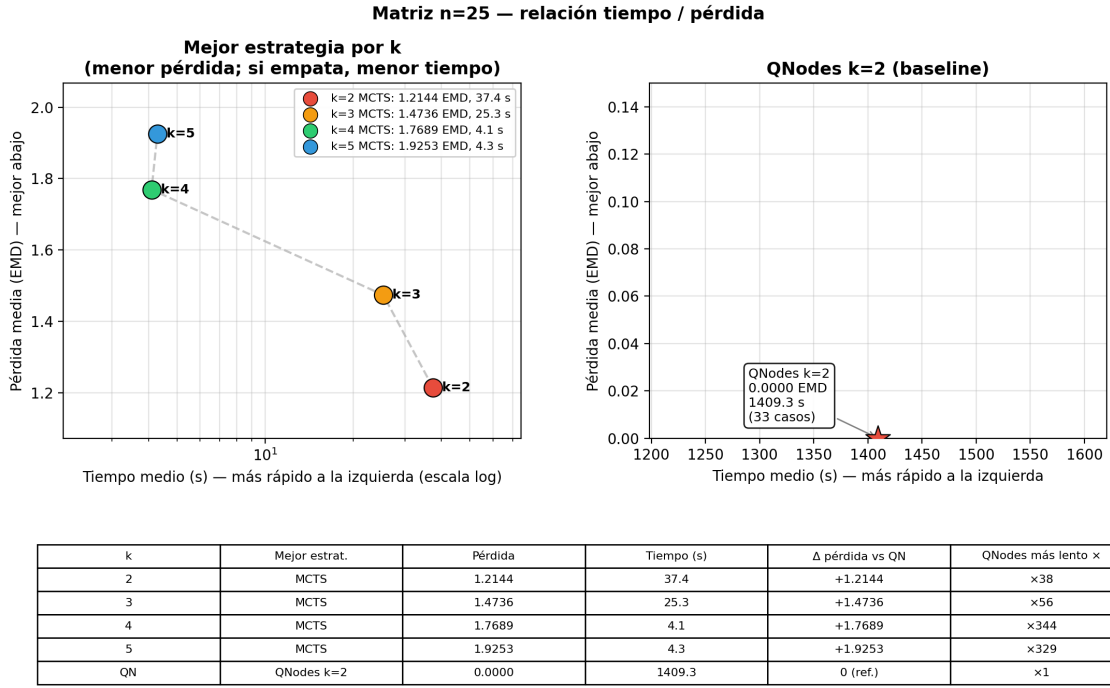
Fuente: n20_comparativa.xlsx · Paneles con ejes independientes (izq. heurísticas k=2...5, der. QNodes k=2)

Figura 3: Matriz $n=20$: mejor heurística por k (izq.) y QNodes $k=2$ (der.).



Fuente: n22_comparativa.xlsx · Paneles con ejes independientes (izq. heurísticas k=2...5, der. QNodes k=2)

Figura 4: Matriz $n=22$: mejor heurística por k (izq.) y QNodes $k=2$ (der.; 42/50 casos).



Fuente: n25_comparativa.xlsx · Paneles con ejes independientes (izq. heurísticas k=2...5, der. QNodes k=2)

Figura 5: Matriz $n=25$: mejor heurística por k (izq.) y QNodes $k=2$ (der.; 33/50 casos).

10. Pseudocódigo

10.1. API principal: find_k_partition

```

ALGORITMO ResolverKPartition(condicion, alcance, mecanismo, tpm, k)
1. subsistema, dists_ref <- PrepararSubsistema(...)
2. SI |nodos| < k ENTONCES RETORNAR Inviabile
3. SI forzar_heuristica = "mcts" ENTONCES
    partes <- HeuristicaMCTS(k); RETORNAR Solution(MCTS, MC_EMD(
        partes))
4. SI k = 2 Y sin forzar ENTONCES RETORNAR GeometricSIA(...)
5. SI ruta_KL_MC rapida ENTONCES RETORNAR SolutionKLMC(...)
6. mip <- ConstruirTablaCostos() // find_mip + tabla_transiciones
7. candidatos <- {Greedy, KL, Clustering, Spectral}
8. RETORNAR argmin_phi(candidatos)

```

10.2. Evaluación de partición

```

ALGORITMO EvaluarParticion(partes)
1. subsistema_partido <- KPartir(partes)
2. dist_part <- subsistema_partido.DistribucionMarginal()
3. phi <- sum_i |dist_part[i] - dists_ref[i]|
4. RETORNAR phi

```

10.3. MC-EMD

```

ALGORITMO MC_EMD(partes, S)
1. PARA CADA nodo futuro f:
2.     Muestrear S estados aleatorios del mecanismo a marginalizar
3.     Lookup vectorizado en NCube[f]; promediar ->
       marginal_estimada[f]
4.     phi_hat <- EMD_Efecto(marginal_estimada, dists_ref)
5.     RETORNAR phi_hat      // error  $O(1/\sqrt{S})$ 

```

10.4. MCTS

```

ALGORITMO HeuristicaMCTS(k, n_iter, c_ucb, S)
1. raiz <- asignacion inicial aleatoria de k etiquetas
2. PARA t = 1..n_iter:
3.     n <- SeleccionUCB(raiz)           // maximizar  $Q + c \cdot \sqrt{\ln N / N_i}$ 
4.     n <- Expandir(n)                 // mover un nodo entre
       partes
5.     estado <- RolloutAleatorio(n)
6.     recompensa <- -MC_EMD(estado, S)
7.     Backpropagation(n, recompensa)
8.     RETORNAR mejor hijo de raiz

```

10.5. QNodes (heurística Q, $k=2$)

```

ALGORITMO QNodes(vertices)
1. omega <- {vertices[0]}; delta <- vertices[1..]
2. PARA cada fase del algoritmo Q:
3.     MIENTRAS |delta| > 1:
4.         PARA cada delta candidato:
5.             phi_union, phi_delta <- FuncionSubmodular(delta,
omega)
6.             Agregar a omega el delta con menor (phi_union - phi_delta
)
7.     Registrar particion candidata en memoria_particiones
8.     RETORNAR particion con minima phi global

```

11. Limitaciones

1. **Frontera exponencial:** cualquier método que marginalice sobre 2^n estados se vuelve inviable para subsistemas grandes; QNodes exacto no completó 50/50 en $n=22$ y $n=25$ dentro del plazo del proyecto.
2. **$k \geq 3$ sin optimalidad garantizada:** Greedy, KL, MCTS y clustering son heurísticas; ϕ reportada es cota superior, no óptimo global.
3. **MC-EMD:** introduce error estadístico $O(1/\sqrt{S})$; no es comparable directamente con ϕ exacta sin margen de tolerancia.
4. **Sin monotonía en k :** $\phi(k=3) \leq \phi(k=2)$ no se garantiza; cada k es búsqueda independiente.

5. **Benchmark exacto $n=25$:** no se completó; la comparativa para $n=25$ usa solo columnas del modo rápido.
6. **Dependencia del hardware:** timeouts y casos omitidos reflejan límites de una estación de trabajo (WSL), no del algoritmo en abstracto.

12. Conclusiones

1. Se extendió exitosamente el repositorio base de QNodes y GeoMIP hacia **k-particiones** ($k = 2, 3, 4, 5$) mediante la clase `KPartitionSIA`, cumpliendo la API `find_k_partition` del enunciado.
2. Se demostró que ϕ (EMD efecto sobre marginales) es la métrica unificadora del proyecto: producto tensorial de Kronecker para reconstruir distribuciones independientes, marginalización sobre N-Cubos para evaluar cada candidato.
3. El benchmark **exacto** cubrió $n \leq 22$ con Geo y Greedy/KL; QNodes $k=2$ coincidió con Geo en todos los casos ejecutados ($n \leq 20$ completo; $n=22$ parcial 42/50).
4. La frontera computacional en $n \geq 20$ se abordó con **MC-EMD** y **MCTS/KL**, permitiendo 50/50 casos en el benchmark rápido hasta $n=25$, distinguiendo explícitamente resultados exactos de aproximados.
5. Las gráficas `matriz_n{n}.png` (Figuras 1–5) sintetizan el trade-off tiempo- ϕ por matriz: QNodes $k=2$ como baseline de calidad frente a la mejor heurística disponible en cada $k \in \{2, 3, 4, 5\}$.
6. El proyecto documenta de forma reproducible scripts, bitácoras, 48 tests y material de sustentación, dejando como trabajo futuro la optimización de `construir_subsistema` (p.ej. vectorización con `bincount`) y la ejecución de los casos QNodes restantes en hardware paralelo.

Referencias del repositorio

- Enunciado: Información para el Proyecto/Proyecto_KQMIP.md
- Manual técnico: docs/Manuales/Manual_Tecnico_GeoMIP_EMD.txt
- Manual de sustentación: documentacion-sustentacion-kqgmip/MANUAL_SUSTENTACION_KQMIP.
- Bitácoras: bitacoras/00_indice.txt (entradas 14, 23, 35, 40, 42)
- Resultados: GeoMIP/data/results/comparativa/n{n}_comparativa.xlsx
- Gráficas: outputs/plots/proyecto/n{n}/matriz_n{n}.png